

Hi, my friend, I want to build a grocery shopping comparison tool. That compares products across different supermarkets. (Pick n Pay, Spar, Checkers, Woolworths), this is an md file he made...help us fix and brainstorm how we can approach things first lets discuss whats in the file

Good context — confirmed my suspicion: none of Checkers, Pick n Pay, Spar, or Woolworths have public product APIs. The price comparisons you see from outlets like BusinessTech and Daily Investor are done by manually checking prices on each retailer's site — Daily Investor built a basket of breakfast ingredients and tracked prices through the retailers' websites or apps, noting that prices for the same product like a 5kg bag of maize meal can differ noticeably across Checkers, Spar, Pick n Pay, and Woolworths. That's a big deal for your project, so let's dig into the doc with that reality in mind.

Here's my read on your friend's scope doc, section by section, and what needs to change for a SA grocery-specific version:

What translates well as-is:

- The overall structure (MVP → Phase 2 → Phase 3) is sound and doesn't need to change.
- Comparison view, product detail pages, "Buy Now" external links — all map directly to grocery.
- The tech stack (React/Next.js, Node/FastAPI, PostgreSQL + Redis) is fine for this too.
- Roles & Responsibilities, Definition of Done, Risks table structure — reusable as-is.

What needs to change — and this is the big one:

Section 6 ("Data Approach for Store Aggregation") assumes official APIs or affiliate feeds exist. For general e-commerce (Amazon, eBay) that's realistic. For SA grocery retailers, it isn't:

- None of the four publish a public product/pricing API.
- No affiliate feeds for grocery baskets that I'm aware of.
- That leaves **web scraping** as effectively your only data source — which was listed as an "option" in the original doc but is really your *primary* strategy here.

That changes your risk profile a lot:

- Grocery sites (especially Checkers Sixty60, Woolworths, PnP ASAP) are dynamic/app-driven, often behind logins for delivery-area selection, and may have anti-bot measures.
- Terms of Service for these retailers likely restrict scraping — worth an actual read before building, not just “check TOS carefully” as a bullet point.
- Prices vary by store location/region in SA (not just online vs in-store), which the original doc doesn’t account for at all.
- Promotions/specials are a huge part of SA grocery shopping culture — the doc’s “Price History / Price Alerts” phase-2 features become much more central for groceries than for general retail.

Other SA-specific gaps in the doc:

- Currency/locale: ZAR, not mentioned.
- Loyalty programs (Pick n Pay Smart Shopper, Checkers Xtra Savings, Woolworths WRewards) affect real prices — do you compare list price or loyalty price?
- Basket-level comparison (a whole shopping list vs one product) is arguably more valuable to your users than single-product comparison — the original doc is single-product-first.
- Load-shedding / hosting reliability isn’t mentioned but could matter if you’re scraping on a schedule from SA infrastructure.

Want to go through these one at a time — starting with the data sourcing/scraping strategy, since that’s the part most likely to make or break the whole project — or would you rather I first rewrite the doc section-by-section with SA grocery specifics baked in?

come up with a plan that is likely to work

Good news and bad news: PriceCheck.co.za already does exactly this using grocery basket data, and it’s had to solve the same data problem you’re about to face. That’s actually useful — it tells us the category is viable, and it tells us how they get around the “no API” problem: careful, targeted data collection focused on a defined basket rather than a full catalog. Here’s a plan built around that reality.

The core strategic bet: narrow scope wins here, not breadth.

Your friend’s original doc assumes “aggregate everything, add stores over time.” For SA groceries, trying to mirror entire catalogs across 4 retailers with 2

developers and no APIs will collapse under its own weight. The realistic path is: **start with a curated basket of ~150–300 high-frequency staple items, matched by barcode/brand/size, refreshed on a schedule — not real-time, not full-catalog.**

Why this works:

- Grocery prices don't move hourly. Daily or twice-daily scraping is completely fine, which massively de-risks the “will we get blocked” problem — low request volume looks nothing like an attack.
- Staples (maize meal, milk, bread, eggs, rice, oil, tea, sugar) are exactly the products that exist as directly comparable SKUs across all four stores — unlike fresh produce or ready meals, which don't match cleanly.
- This mirrors what PriceCheck itself does for its grocery index (a defined ~29-item basket), and what every journalist price-war article does — it's the proven format in this market.

Data sourcing approach, ranked by risk:

1. **Structured product data (JSON-LD/schema.org) published on the retailer sites for Google Shopping/SEO purposes** — lowest legal/technical risk, since it's explicitly published for machine consumption. Check this first for each retailer before anything else.
2. **The retailers' own delivery-app web endpoints** (Sixty60, Asap!, Dash, Spar2U) — these are what journalists already use to get per-item online prices, but you're calling internal APIs not meant for third parties, so treat this as higher-risk: low request volume, no scraping at login-gated depth, and be ready to stop if asked.
3. **Full headless-browser scraping of rendered pages** — last resort, most brittle, most likely to break or get blocked.

Do a **1-week technical spike before writing any product code**: for each of the 4 retailers, check robots.txt, look for structured data, and test-fetch ~20 products. This tells you which retailers are “easy,” which are “hard,” and whether your MVP should launch with 2 stores or 4 (starting with the 2 easiest is fine — Checkers and Pick n Pay are the most-covered in existing price comparisons, so likely the most scrapeable too).

Product matching — the other big risk from the original doc: Match by **barcode (EAN) where retailers expose it, otherwise brand + product name + pack size**, curated by hand for your initial ~200 items rather than built with

fuzzy-matching automation. Manual curation for a small basket is completely tractable for 2 people and avoids the “is this the same product” mess that sinks most comparison tools.

Revised phase structure:

12-week phased plan for an SA grocery price comparison MVP

1. Week 1: Data feasibility spike

Both of you, together. For Checkers, Pick n Pay, Spar, Woolworths: check robots.txt, look for schema.org/JSON-LD product data on category pages, and test-fetch pricing for ~20 known staple items (test via Sixty60/Asap!/Dash/Spar2U where needed). Rank each store easy/medium/hard. Decide your launch set of 2 stores based on results, not preference.

2. Weeks 2-3: Curated basket + schema

Hand-pick ~150-300 staple items that exist at all launch stores (maize meal, milk, bread, eggs, rice, oil, tea, sugar, canned goods). Build the Postgres schema around barcode-first matching with brand/size fallback. This list becomes your entire MVP catalog - resist the urge to widen it yet.

3. Weeks 4-7: Scraper + normalization pipeline

Build scheduled (daily, not real-time) scrapers per store feeding a raw staging table, then a normalization job that maps raw entries to your curated product list. Cache aggressively - the frontend never scrapes live. Add basic anomaly checks (e.g. flag a price that jumped 300% - probably a bad scrape, not a real change).

4. Weeks 8-10: Frontend - basket comparison first

Build the ‘add items to a list, see total cost per store’ basket comparison as the hero feature, before single-product comparison pages. This is the more valuable and more differentiated experience for grocery vs the general product-comparison template, and it’s what every SA price-war article proves people already care about.

5. Weeks 11-12: QA, legal check, soft launch

Read each retailer’s actual Terms of Service (not just skim), add a clear disclaimer that prices are indicative and sourced independently, remove any store logos/trademarks you don’t have rights to use, and launch quietly to friends/family. Be ready to adjust or drop a store if you get a takedown request rather than treating it as a crisis.

6. Month 4+: Widen, don't rebuild

Once the pipeline is proven on 150-300 items and 2 stores, expand the curated list and add the other 2 stores using the same pipeline. Add price history (you're already scraping daily, so this is nearly free), loyalty-price toggle (Smart Shopper/Xtra Savings/WRewards vs list price), and specials tracking - this is genuinely differentiated versus PriceCheck's static basket approach.

A few things worth flagging on top of the plan:

- **Spar is franchised** — prices genuinely differ store-to-store, unlike the other three. You'll need to decide early whether you support location-based Spar pricing or just pick one representative store/region for MVP.
- **Your realistic competitor isn't PriceCheck's whole platform, it's their grocery basket feature** — they update it periodically as a news hook, not as a live, continuously-updated product. A daily-refreshed, always-on basket comparison tool would genuinely be a step up from what exists today.
- **Don't build user accounts or price alerts in the MVP**, even though they're tempting — they only matter once your data pipeline is trustworthy, and trust is the thing you're least sure of right now.

Want me to rewrite the full scope doc with this plan baked in as a new version, or start with the Week 1 spike checklist (robots.txt / structured-data check) in more technical detail?

re write a full scope

grocery-comparison-scope.md

Project Scope: SA Grocery Price Comparison Tool

1. Project Overview

Project Name: (TBD — e.g., “BasketCompare” / “TrolleyCheck”) **Team:** 2 developers **Type:** Web Application **Goal:** Build a tool that lets South African shoppers compare grocery prices — starting with a curated basket of everyday

staples — across Pick n Pay, Checkers, Spar, and Woolworths, so they can see which store gives them the cheapest overall shop.

Key difference from a generic product-comparison site: there are no public APIs or affiliate feeds for SA grocery retailers. The whole plan is built around that constraint — narrow, curated, low-frequency data collection rather than full-catalog aggregation.

2. Objectives

- Let users build a shopping list and see the total cost of that list at each store
 - Compare prices on a curated set of common grocery staples, matched accurately across stores (barcode/brand/size — not fuzzy name-matching)
 - Surface which store is cheapest overall and per category
 - Track price history over time (near-free once daily scraping exists)
 - Expand store and product coverage gradually, once the pipeline is proven — not before
-

3. Target Audience

- Budget-conscious households doing a weekly/monthly big shop
 - Shoppers comparing loyalty-program value (Smart Shopper, Xtra Savings, WRewards) vs list price
 - People deciding which store to shop at before leaving home, not mid-aisle comparison
-

4. Competitive Context

PriceCheck.co.za already publishes a periodic grocery basket comparison (~29 items across Checkers, Pick n Pay, Woolworths) as a news/content feature, not a live, continuously-updated tool. Media outlets (BusinessTech, Daily Investor, Netwerk24) do similar one-off manual comparisons.

The gap this project fills: none of these are a live, always-on, basket-builder tool that updates daily and lets a user plug in *their own* shopping list rather than a fixed sample basket. That's the differentiation to build toward — not full catalog breadth.

5. Core Features

MVP

Feature	Description
Curated Staple Catalog	~150–300 hand-picked items (maize meal, milk, bread, eggs, rice, oil, tea, sugar, canned goods, etc.) matched by barcode or brand+size across launch stores
Basket Builder	User adds items to a list; see total cost per store side by side
Single-Item Comparison	Price per store for one product, as a secondary view
Store Coverage	Launch with 2 stores (whichever come out “easy” from the Week 1 data spike), not all 4
Daily Price Refresh	Scheduled scraping (daily/twice-daily), not real-time
Responsive Design	Mobile-friendly — most SA grocery shopping research happens on phones
Data Disclaimer	Clear “prices are indicative, independently sourced” messaging

Phase 2 (Post-MVP)

Feature	Description
Full 4-Store Coverage	Add remaining stores once the pipeline is proven
Expanded Catalog	Grow beyond staples using the same barcode/brand matching approach
Price History	Chart price trends per item/store using accumulated daily scrapes
Loyalty Price Toggle	Show list price vs Smart Shopper/Xtra Savings/WRewards price

Feature	Description
Specials Tracking	Flag current promotions per store
Spar Regional Handling	Decide and implement how franchised, location-varying Spar pricing is represented (single reference store vs location-aware)
User Accounts	Save baskets/shopping lists
Price Alerts	Notify when a saved basket's cheapest store changes or a tracked item drops

Phase 3 (Growth)

Feature	Description
Browser Extension	Compare while browsing a retailer's site directly
More Retailers	Boxer, Food Lover's Market, Makro
Public API	For developers who want the normalized price data
ML-Assisted Matching	Reduce manual curation burden as catalog scales past hand-matched territory

6. Out of Scope (v1)

- Processing payments or fulfilling orders
 - Fresh produce / non-SKU'd items (weight-variable, hard to match consistently across stores)
 - Native mobile apps
 - Real-time pricing (daily refresh is the target, not live)
 - Full catalog parity with any single retailer
 - Location-aware pricing beyond a single reference point per store (Phase 1)
-

7. Technical Architecture

Stack

Layer	Technology
Frontend	Next.js (React + Tailwind)
Backend	Node.js + Express or Python + FastAPI
Database	PostgreSQL (products, stores, prices, price history) + Redis (serving cache)
Scraping/Ingestion	Scheduled jobs (cron/queue), headless browser only where structured data isn't available
Hosting	Vercel (frontend) + Railway/Render (backend + scheduled jobs)
Monitoring	Sentry (errors), Plausible/GA (analytics)
Auth (Phase 2)	Clerk, Auth0, or Supabase Auth

High-Level Architecture

Scheduled Scrapers (per store)



Raw Staging Table (Postgres)



Normalization Job

(barcode / brand+size match → curated product list)



Products & Prices (Postgres) ← price history accumulates here



Redis cache



Backend API → Frontend (Next.js) → User

The frontend and API never scrape live. All scraping is scheduled and decoupled from user requests — this keeps request volume to each retailer low and predictable, which matters both for reliability and for staying a well-behaved, low-footprint visitor.

Data Sourcing Strategy — Ranked by Risk

1. **Structured product data (schema.org/JSON-LD)** published on retailer sites for Google Shopping/SEO — lowest risk, explicitly machine-readable, check this first for every store.
2. **Retailer delivery-app web endpoints** (Sixty60, Asap!, Dash, Spar2U) — what journalists already use for online per-item pricing; treat as higher risk since these aren't published for third-party use. Keep request volume low, don't bypass login/paywalls, be ready to stop if asked.
3. **Full page scraping via headless browser** — last resort; most brittle and most likely to be blocked or break on redesign.

Product Matching

- **Primary key:** barcode (EAN), where exposed
 - **Fallback:** brand + product name + pack size, curated manually for the initial ~150–300 item list
 - Manual curation is deliberate for MVP — automated fuzzy matching is a Phase 3 problem, not a Week 1 one
-

8. Project Phases & Milestones

Phase 0 — Data Feasibility Spike (Week 1)

- ☐ Check robots.txt for each of the 4 retailers
- ☐ Check for schema.org/JSON-LD structured product data on each site
- ☐ Test-fetch pricing for ~20 known staple items per store (via delivery apps where needed)
- ☐ Rank each store easy/medium/hard based on findings
- ☐ Decide the 2 launch stores based on results, not preference
- ☐ Read the actual Terms of Service for the launch stores (not just skim)

Phase 1 — Foundation (Weeks 2–3)

- ☐ Finalise name, domain, branding
- ☐ Set up GitHub repo, project board
- ☐ Hand-pick the 150–300 item curated staple list, shared across launch stores
- ☐ Design database schema around barcode-first matching
- ☐ Wireframe the 3 core pages: home, basket builder, comparison results
- ☐ Set up CI/CD

Phase 2 — MVP Build (Weeks 4–10)

- ☐ Build scheduled scrapers per launch store → raw staging table
- ☐ Build normalization job mapping raw data to curated product list
- ☐ Add anomaly detection (flag implausible price jumps as likely bad scrapes)
- ☐ Build backend API (basket total, single-item comparison endpoints)
- ☐ Build frontend: basket builder as the primary experience, single-item comparison secondary
- ☐ Add Redis caching for served results
- ☐ Unit tests for normalization and matching logic (this is the highest-risk code in the project)

Phase 3 — MVP Launch (Weeks 11–12)

- ☐ QA across devices/browsers
- ☐ Performance audit (Lighthouse)
- ☐ Error monitoring (Sentry) and analytics live
- ☐ Add data disclaimer and remove any unlicensed use of store logos/trademarks
- ☐ Soft launch to friends/family
- ☐ Have a plan for responding to a takedown/access request from a retailer

Phase 4 — Iteration (Month 4+)

- ☐ Add remaining 2 stores using the proven pipeline
 - ☐ Expand curated catalog beyond staples
 - ☐ Price history charts
 - ☐ Loyalty price toggle
 - ☐ Specials tracking
 - ☐ Decide and implement Spar's location-variable pricing approach
 - ☐ User accounts and saved baskets
-

9. Roles & Responsibilities

Area	Owner	Notes
Scraping & Data Ingestion	Shared, front-loaded	Highest-risk area — do the Week 1 spike together
Normalization &	Person A	Barcode/brand/size

Area	Owner	Notes
Matching		matching logic, anomaly checks
Backend API	Person B	Endpoints, caching, business logic
Frontend	Person B (or shared)	Basket builder, comparison views
DevOps / Infra	Rotate	Hosting, scheduled jobs, monitoring
Legal/Compliance Check	Shared	ToS review, disclaimer wording, trademark use

10. Definition of Done

A feature is complete when it:

- Passes all written tests (especially matching/normalization logic)
 - Has been reviewed in a pull request by the other person
 - Is deployed to staging and manually tested against real scraped data
 - Is documented (inline comments + README update)
-

11. Key Risks & Mitigations

Risk	Likelihood	Mitigation
Scraping blocked or ToS issue	High	Start with structured/SEO data where possible; keep frequency low; be ready to drop a store without treating it as a crisis
Product mismatch across stores	High	Barcode-first matching; manual curation for MVP scope; anomaly checks on price jumps
Spar's franchised,	Medium	Explicitly scoped to

Risk	Likelihood	Mitigation
location-varying pricing		Phase 2; use single reference store for MVP
Site redesigns breaking scrapers	Medium	Keep scrapers isolated per store; monitor for silent failures, not just errors
Scope creep (full catalog temptation)	Medium	Lock the curated list size for MVP; log expansion ideas for Phase 2
Burnout (2-person team)	Medium	Time-box phases; the Week 1 spike is a hard go/no-go checkpoint, not just a milestone

12. Success Metrics

Metric	MVP Target	6-Month Target
Stores covered	2	4
Curated items covered	150–300	1,000+
Data refresh reliability	Daily, >90% success rate	Daily, >98% success rate
Monthly active users	50 (friends/family)	1,000+
Basket comparisons run	—	Track as core engagement metric

13. Next Immediate Actions

1. **This week:** Run the Phase 0 data feasibility spike — this determines your launch store pair and whether the plan needs adjusting before any product code is written
2. **This week:** Read the Terms of Service for all 4 retailers, not just the 2 you expect to launch with
3. **Next week:** Hand-pick the curated staple list together
4. **Next week:** Design the barcode-first database schema before writing scraper code

Document version: 2.0 — SA grocery-specific rewrite, data-strategy-led. Update as the Week 1 spike results come in. Rewrote it as v2.0, data-strategy-led rather than generic e-commerce-led. The Week 1 spike is treated as a real go/no-go checkpoint — its results should decide your launch store pair before you write any product code.

what do you think about supermarket wizard

On the name itself: “Supermarket Wizard” is free and clear as far as I can find — no existing SA app or brand using it. My honest take on it as a name:

- It’s clear and easy to say/remember, which matters
- “Wizard” leans a bit generic/utility-app-vibes (setup wizards, tax wizards) rather than evoking “saving money” or “smart shopping” — it doesn’t say *why* someone would use it
- “Supermarket” is accurate but a bit stiff/corporate next to friendlier competitor names in this space

But there’s something more important than the name I need to flag: while checking, I found a direct competitor already moving into exactly this space.

A new app called Grocify launched an invite-only preview in South Africa this year — a mobile app comparing grocery prices from Checkers, Pick n Pay, Spar, and Woolworths in one place, built by Johannesburg developer Ethan Stander, pulling near real-time pricing from all four retailers. It uses the user’s location to show prices from the four nearest stores per retailer, and full public release was expected before the end of February.

That’s essentially your exact concept, already built, already live in preview, from a solo SA developer. Given today’s date, it’s likely already public.

This matters for your plan:

- It doesn’t mean don’t build it — but it changes the calculus on differentiation. Near-real-time full catalog is apparently what Grocify is going for; your curated-basket, matched-by-barcode, basket-total-comparison approach could actually be a smarter, more defensible niche than trying to out-build a live competitor on breadth.

- Worth actually downloading/trying Grocify before you write more code, to see what it does well and where the gaps are — that's now genuinely useful competitive research, not just theoretical risk.

Want me to search for more on Grocify specifically (reviews, what it covers, any public feedback) before you two commit further?